

Evelyn Jeaneth Guarchaj Tahay 202408041  
Ing.Sistemas y Ciencias de la Computación “D”  
Ing.José Pablo Sánchez Estrada  
Arquitectura de Sistemas I  
Agosto, 2026



### API con 4 métodos

1. Iniciar el proyecto Node.js e instalar las librerías de TypeScript, hono y pino.



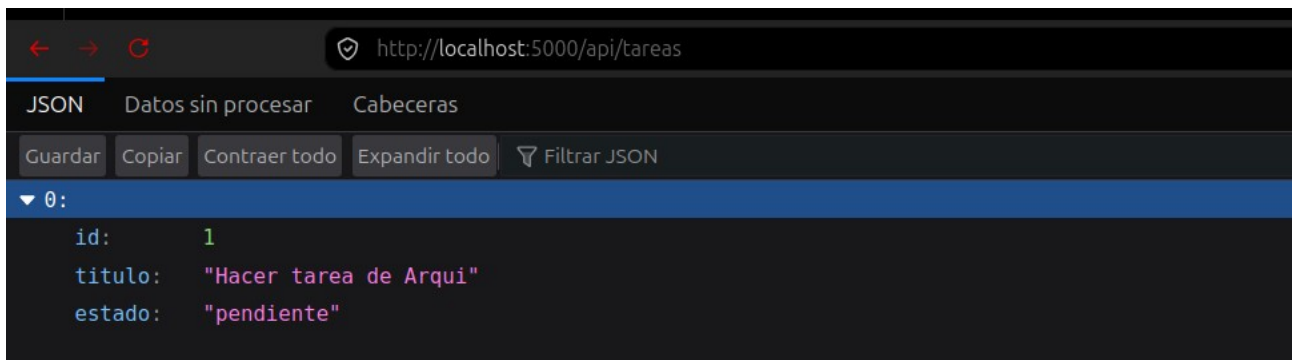
2. Crear archivo index.ts en dónde irá todo el código en TypeScript



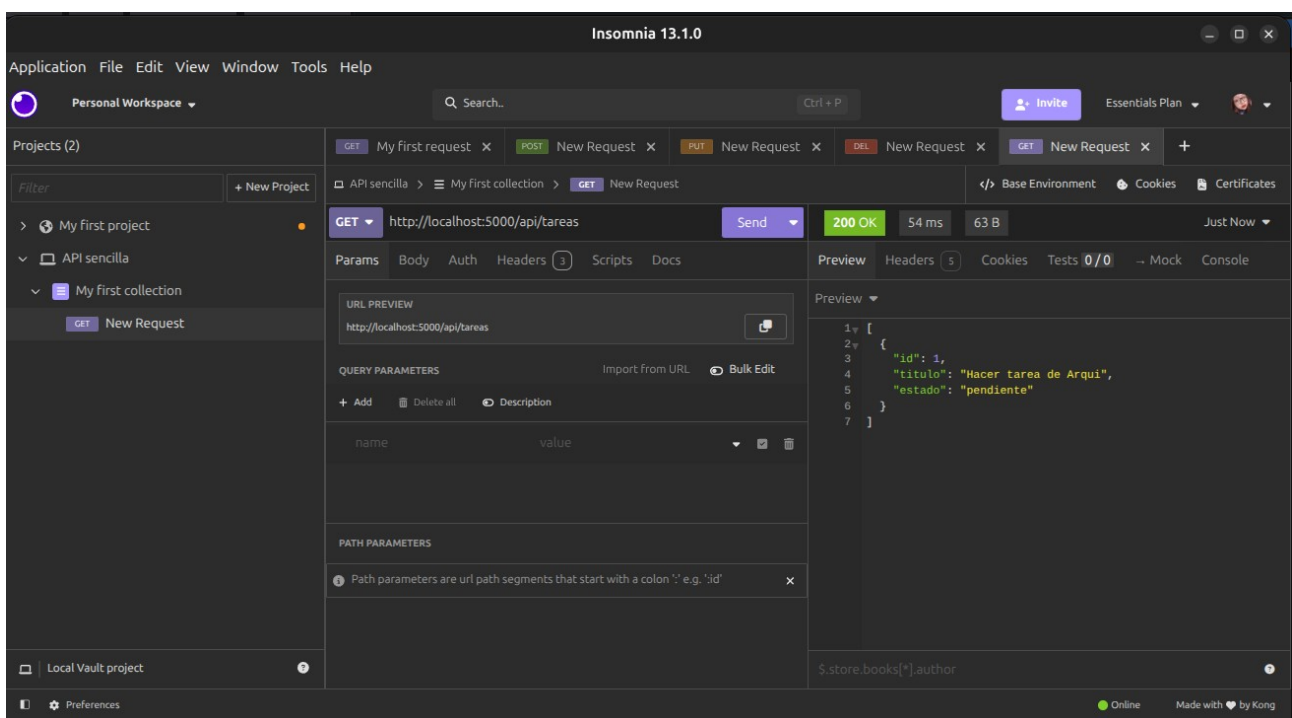
3. El método **GET**: Indica que si alguien entra a la ruta de la api usando el método GET, toma la lista de tareas, la convierte en JSON y la envía hacia donde se hizo el request junto con el código 200.

```
// GET codigo validacion
app.get('/api/tareas', (c) =>{
  return c.json(tareas,200)
})
```

Captura de la respuesta JSON del endpoint /api/tareas (método GET) visualizada directamente en el navegador Firefox mediante su visor nativo de datos JSON



Petición realizada desde Insomnia:

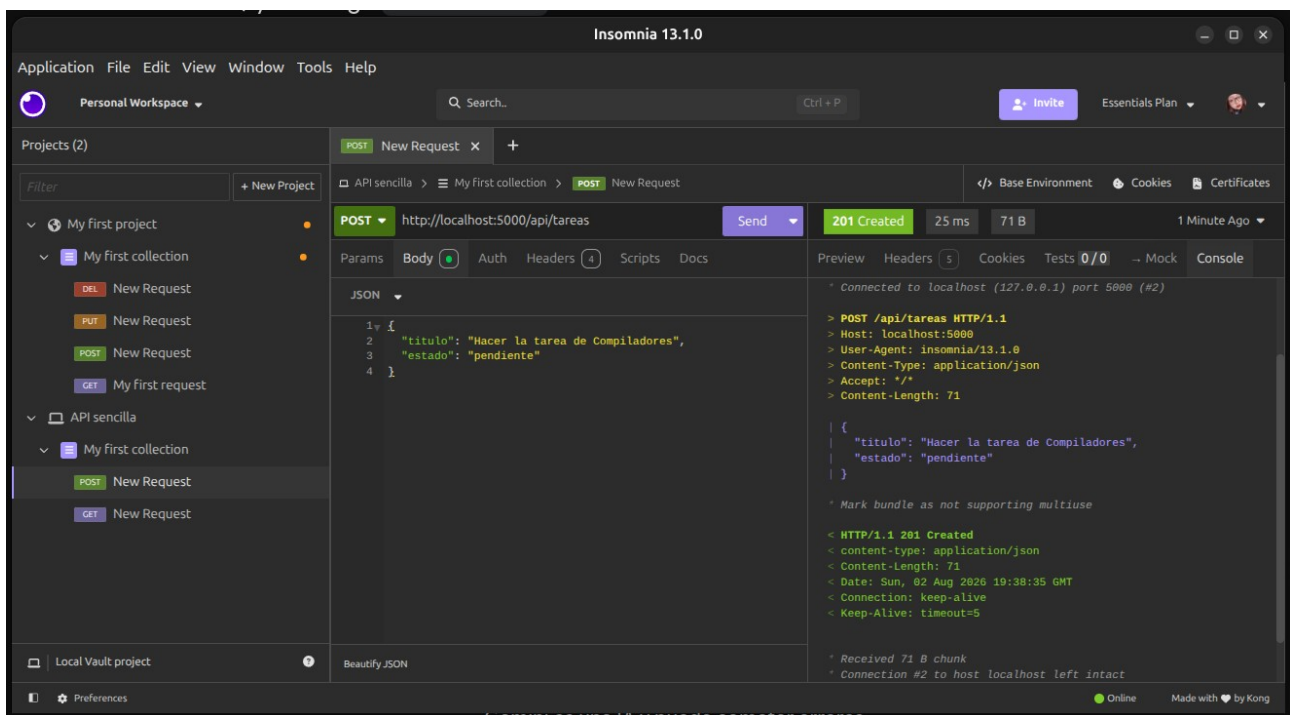


4. Método **POST**: En este método se toma el texto JSON que se envía desde la pestaña Body de Insomnia y se convierte en un objeto de TypeScript de forma asíncrona, se calcula un nuevo ID midiendo el tamaño del arreglo de tareas existentes y sumándole 1, además de asignar valores por defecto en caso de que falte algún dato. Por último, se guarda la nueva tarea en el arreglo en memoria y se devuelve la tarea creada en formato JSON junto con el código de estado HTTP 201 created para indicar que se creó exitosamente.

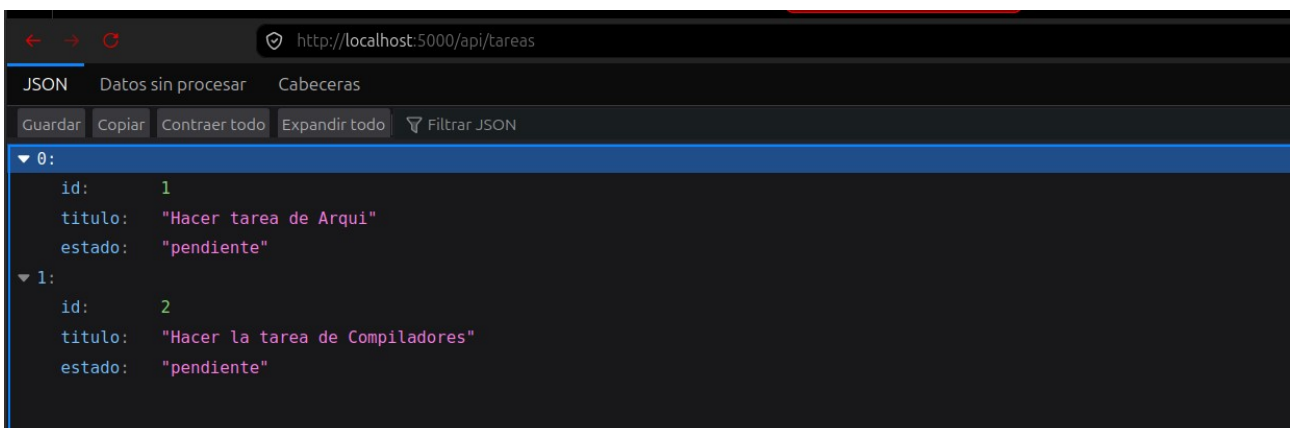
```
//POST codigo creacion
app.post('/api/tareas', async (c) => {
  const nuevaTarea = await c.req.json<Partial<Tarea>>()

  const tareaCreada: Tarea = {
    id: tareas.length + 1,
    titulo: nuevaTarea.titulo || 'Sin titulo',
    estado: nuevaTarea.estado || 'pendiente'
  }

  tareas.push(tareaCreada)
  return c.json(tareaCreada, 201)
})
```



Y en el visor de JSON se puede ver como la tarea ya fue creada:



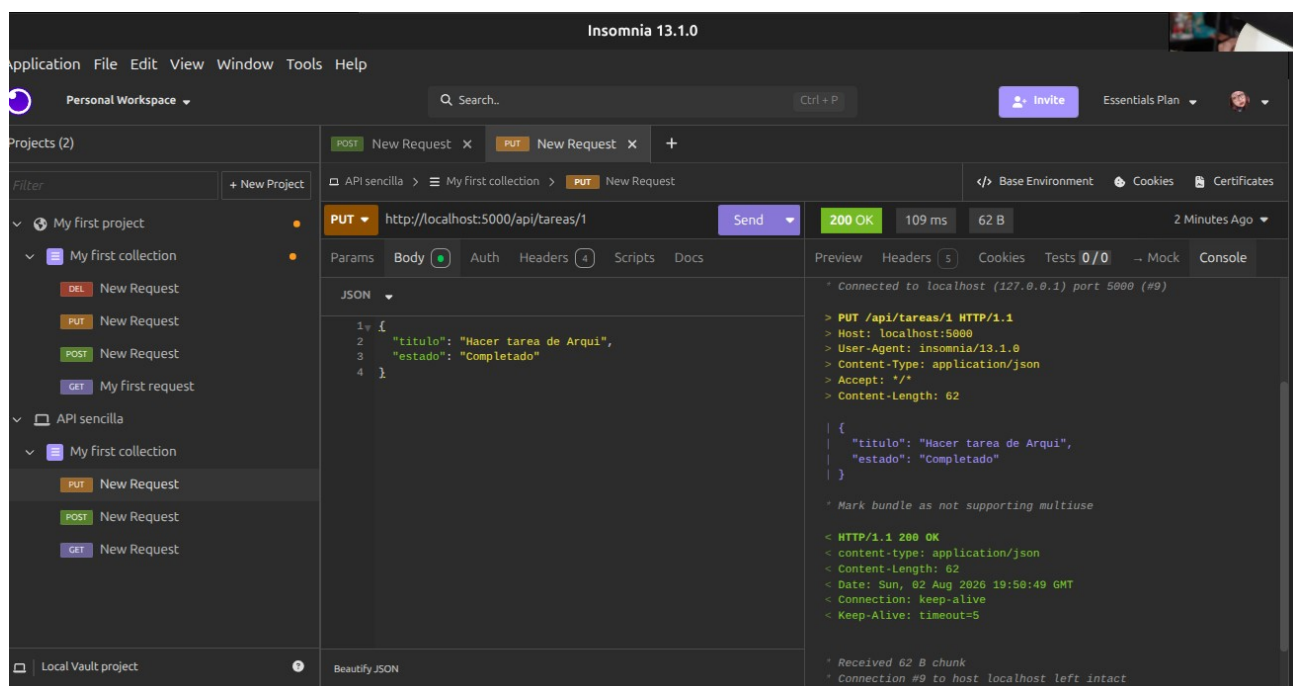
5. Método **PUT**: Este método le indica a la ruta que debe esperar un identificador al final de la URL (:id), el cual se extrae y convierte en número entero para saber qué registro editar. Se capturan los datos en formato JSON enviados desde Insomnia de forma asíncrona. En lugar de recorrer la lista con un ciclo clásico, se utiliza el método de arreglo .find() en TypeScript para localizar de forma directa la tarea que coincida con el ID proporcionado. Si la encuentra, actualiza el título y el estado manteniendo los datos originales si no se envían cambios (usando el operador ??), devolviendo la tarea actualizada con un código 200 OK. Si no encuentra una tarea con ese ID, el servidor devuelve un mensaje de error y el código HTTP 404 Not Found.

```
//PUT
app.put('/api/tareas/:id', async (c) => {
  const id = Number(c.req.param('id'))
  const datosNuevos = await c.req.json<Partial<Tarea>>()

  const tarea = tareas.find((t) => t.id === id)
  if (!tarea){
    return c.json({ error: 'Tarea no encontrada '}, 404)
  }

  tarea.titulo = datosNuevos.titulo ?? tarea.titulo
  tarea.estado = datosNuevos.estado ?? tarea.estado

  return c.json(tarea, 200)
})
```



Y en el visor de JSON se puede ver como la tarea 1 ya fue actualizada:

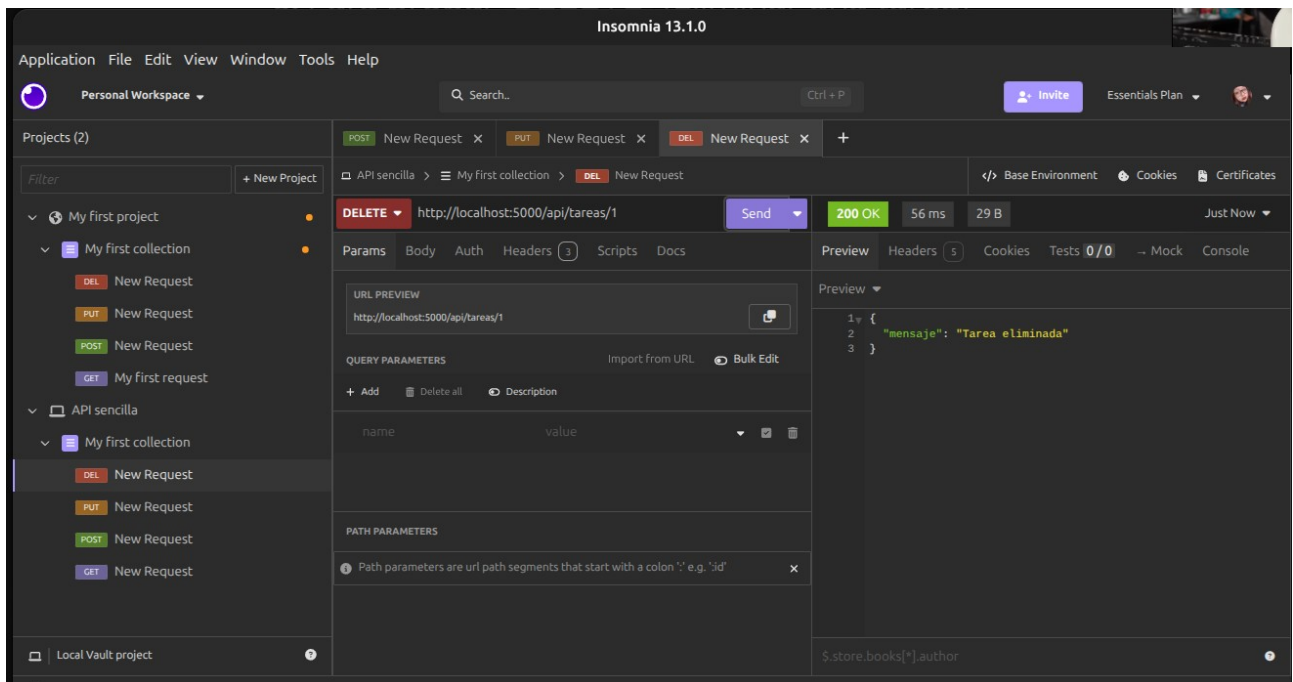


8. Método **DELETE**: La lógica es idéntica a la de PUT, busca por ID en la URL. Si la encuentra borra ese registro de la lista.

```
// DELETE
app.delete('/api/tareas/:id', (c) => {
  const id = Number(c.req.param('id'))
  const indice = tareas.findIndex((t) => t.id === id)

  if(indice === -1) {
    return c.json({ error: 'Tarea no encontrada' }, 404)
  }

  tareas.splice(indice, 1)
  return c.json({ mensaje: 'Tarea eliminada' }, 200)
})
```



Y en el visor de JSON se puede ver como la tarea 1 ya fue eliminada:

